

CS-338 Intro to the Theory of Computation

- Lecture #12

➤ **Complexity Theory: P Class**

- Prof Pietro S. Oliveto

Department of Computer Science and Engineering

Southern University of Science and Technology (SUSTech)

- `olivetop@sustech.edu.cn`
<https://faculty.sustech.edu.cn/olivetop>
- Office: Room 113

Overview

Last time:

- The Reducibility Method for proving undecidability and T-unrecognizability.
- General reducibility
- Mapping reducibility

Today: (Sipser §7.1)

- Introduction to Complexity Theory
- Complexity classes; the Class P

Intro to Complexity Theory

Computability theory (1930s - 1950s):

Is A decidable?

Complexity theory (1960s - present):

*Is A decidable with restricted resources?
(time/memory/...)*

Example: Let $A = \{a^k b^k \mid k \geq 0\}$.

Q: How many steps are needed to decide A ?

We need:

- The Turing machine (the algorithm)
- A definition of “*time*”
 - Depends on the input.

We need a way of measuring time...

Time Complexity: running time

- The number of steps an algorithm uses on a problem depends on the input
 - Eg., if the input is a graph it may depend on
 - The number of nodes
 - The number of edges
 - The maximum degree of the graph
 - Or combinations of the above and more
- For simplicity we count the number of steps required as a function of the length of the input
- We give an upper bound for all inputs of length n , called “**worst-case complexity**”.

Def

Let M be a deterministic Turing machine that halts on all inputs. The *running time* or *time complexity* of M is the function $f: \mathcal{N} \rightarrow \mathcal{N}$, where $f(n)$ is the maximum number of steps that M uses on any input of length n . If $f(n)$ is the running time of M , we say that M runs in time $f(n)$ and that M is an $f(n)$ time Turing machine. Customarily we use n to represent the length of the input.

Asymptotic notation

- The runtime $f(n)$ of an algorithm is often a complex expression
 - Eg., $f(n) = 5n^2 + 10n - 10$
- We use **asymptotic notation** to express it in a convenient form
- Idea: capture *asymptotic growth*:
 - Ignore constant factors
 - Ignore small order terms
 - Ignore what happens for small inputs (they are easy anyway)

Example

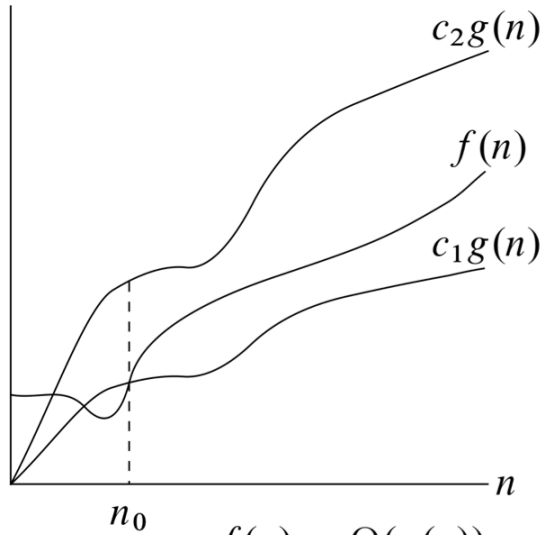
$$5n^2 + 10n - 10 = \Theta(n^2)$$

$$5n^2 + 10n - 10 = O(n^2) = O(n^3)$$

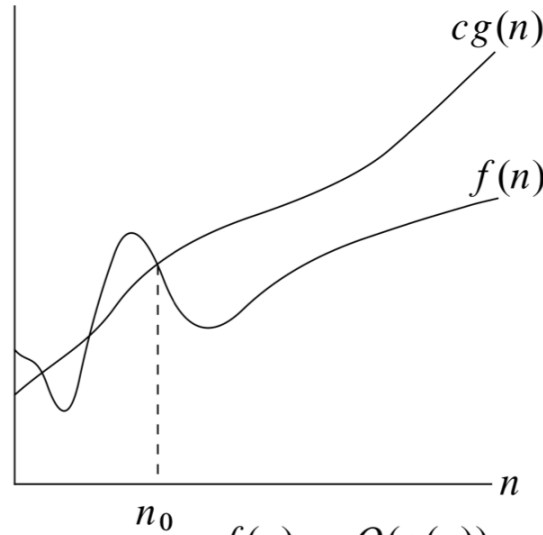
$$5n^2 + 10n - 10 = \Omega(n) = \Omega(n^2)$$

Are all true!

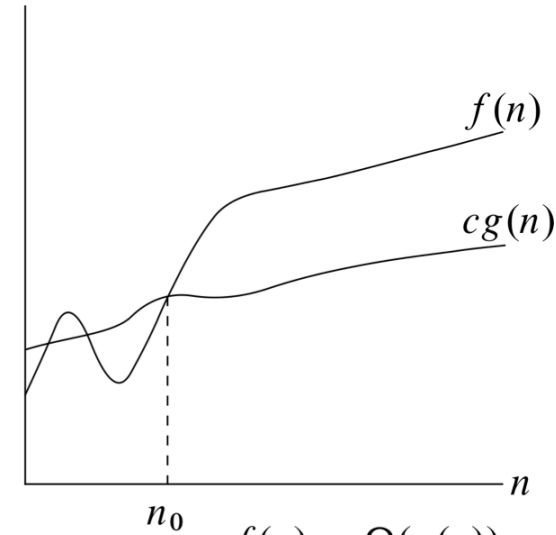
Asymptotic notation



$f(n) = \Theta(g(n))$
(a)



$f(n) = O(g(n))$
(b)



$f(n) = \Omega(g(n))$
(c)

$O(g(n)) = \{f(n) : \text{there exist constants } 0 < c \text{ and } n_0 \text{ such that}$
 $0 \leq f(n) \leq cg(n) \text{ for all } n \geq n_0\}$

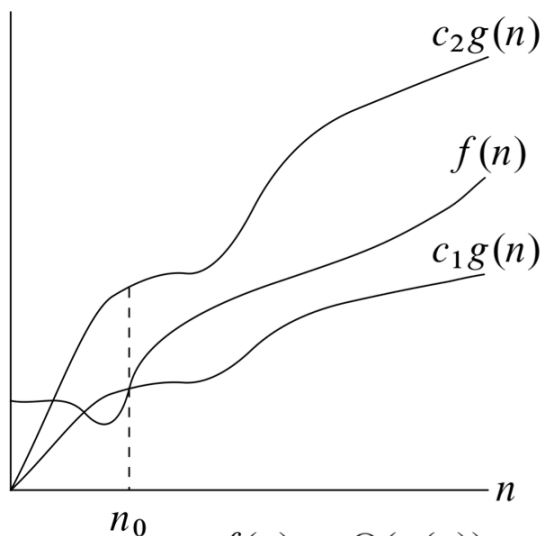
$f(n) = o(g(n))$ if $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$

$\Omega(g(n)) = \{f(n) : \text{there exist constants } 0 < c \text{ and } n_0 \text{ such that}$
 $0 \leq cg(n) \leq f(n) \text{ for all } n \geq n_0\}$

$f(n) = \omega(g(n))$ if $g(n) = o(f(n))$

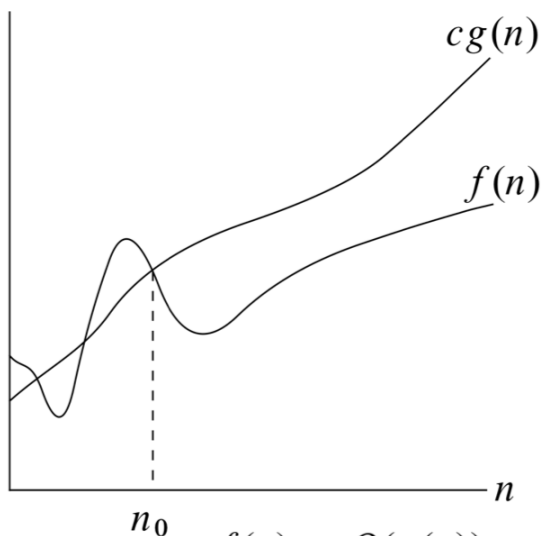
$\Theta(g(n)) = \{f(n) : \text{there exist constants } 0 < c_1 \leq c_2 \text{ and } n_0 \text{ such that}$
 $0 \leq c_1g(n) \leq f(n) \leq c_2g(n) \text{ for all } n \geq n_0\}$

Asymptotic notation: examples



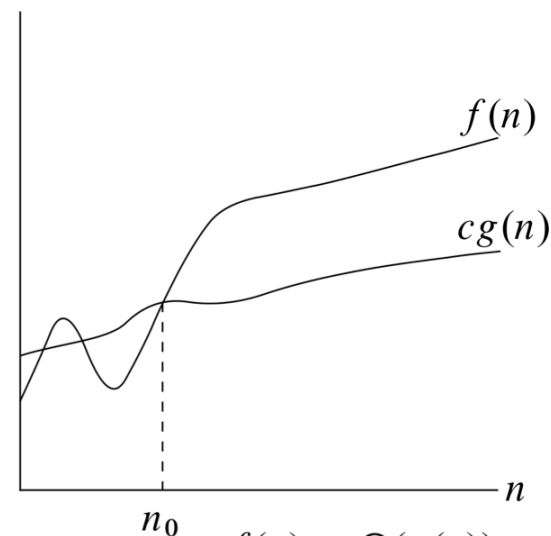
$$f(n) = \Theta(g(n))$$

(a)



$$f(n) = O(g(n))$$

(b)



$$f(n) = \Omega(g(n))$$

(c)

- $2n + 1 = O(n)$
- $42 = O(n)$ (but not $\Theta(n)$!)
- $n - 9 = \Omega(n)$
- $n^2 + n = \Omega(n)$ (but neither $O(n)$, nor $\Theta(n)$!)
- $n^3 = o(n^4) = o(2^n)$
- $\sqrt{n} = \omega(\log n)$

Asymptotic notation: common runtimes

$\Theta(1)$	constant time
$\Theta(\log n)$	logarithmic time
$\Theta(n)$	linear time
$\Theta(n^2)$	quadratic time
$\Theta(n^3)$	cubic time
n^k for $k = \Theta(1)$	polynomial time
2^n	exponential time

- Every polynomial of $\log n$ grows strictly slower than every polynomial of n :
 - Eg., $(\log n)^{100} = o(n^{0.01})$
- Every polynomial of n grows strictly slower than every exponential function 2^{n^ε} :
 - Eg., $n^{100} = o(2^{n^{0.01}})$

The following are easy to check.

1. $\sqrt{n} = o(n)$.
2. $n = o(n \log \log n)$.
3. $n \log \log n = o(n \log n)$.
4. $n \log n = o(n^2)$.
5. $n^2 = o(n^3)$.

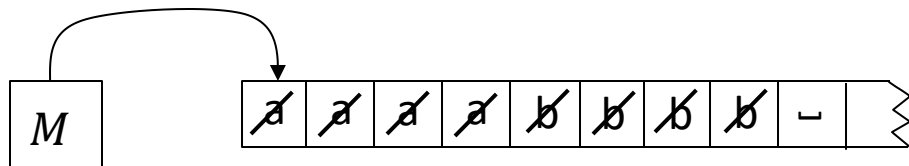
steps to decide $A = \{a^k b^k \mid k \geq 0\}$

Theorem: A 1-tape TM M can decide A where, on inputs of length n , M uses at most cn^2 steps, for some fixed constant c .

Terminology: M uses $O(n^2)$ steps.

Proof: M = "On input w

1. Scan input to check if $w \in a^*b^*$, *reject* if not.
2. Repeat until all crossed off.
Scan tape, crossing off one a and one b .
Reject if only a 's or only b 's remain.
3. Accept if all crossed off. "



- We could cut two symbols at a time:
 - Still $\Theta(n^2)$
- Same holds for any constant number of symbols

Analysis:

$O(n)$ steps
 $+O(n)$ iterations
 $\times O(n)$ steps

 $O(n) + O(n^2)$ steps
 $= O(n^2)$ steps

$\Theta(n)$ steps ($2n$ steps)
 $+ \Theta(n)$ iterations ($n/2$ iterations)
 $\times \Theta(n)$ steps (n steps)

Check-in 12.1

How much improvement is possible in the bound for this theorem about 1-tape TMs deciding A ?

- (a) $O(n^2)$ is best possible.
- (b) $O(n \log n)$ is possible.
- (c) $O(n)$ is possible.

Deciding $A = \{a^k b^k \mid k \geq 0\}$ faster

Theorem: A 1-tape TM M can decide A by using $O(n \log n)$ steps.

Proof:

M = "On input w

1. Scan tape to check if $w \in a^*b^*$. *Reject* if not.
2. Repeat until all crossed off.
Scan tape, crossing off every other a and b .
Reject if even/odd parities disagree.
3. Accept if all crossed off. "

Analysis:

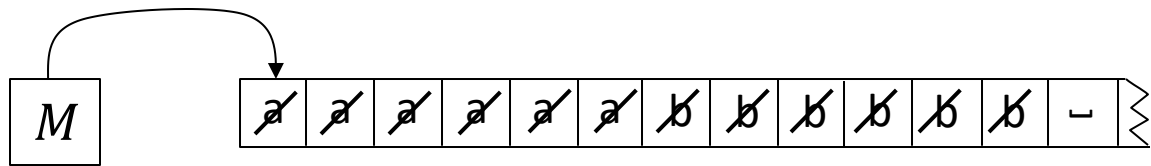
$\Theta(n)$ steps

+ $\Theta(\log n)$ iterations

$\times \Theta(n)$ steps

 $\Theta(n) + \Theta(n \log n)$ steps

$= \Theta(n \log n)$ steps



	Parities		
a's	even (6)	odd (3)	odd (1)
b's	even (6)	odd (3)	odd (1)

Further improvement?

Not possible.

Theorem: A 1-tape TM M cannot decide A by using $o(n \log n)$ steps.

You are not responsible for knowing the proof.

Deciding $A = \{a^k b^k \mid k \geq 0\}$ even faster

Theorem: A multi-tape TM M can decide A using $O(n)$ steps.

M = “On input w

1. Scan input to check if $w \in a^*b^*$, *reject* if not.
2. Copy a ’s to second tape.
3. Match b ’s with a ’s on second tape.
4. *Accept* if match, else *reject*. ”

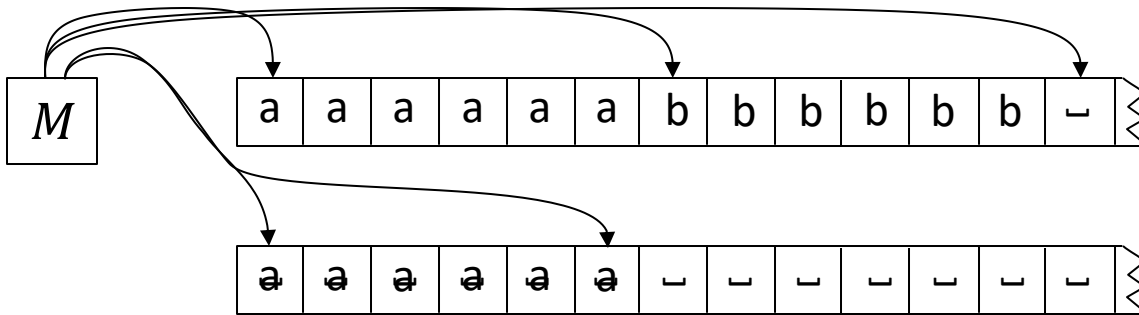
Analysis:

$O(n)$ steps

$+O(n)$ steps

$+O(n)$ steps

$= O(n)$ steps



Model Dependence

Number of steps to decide $A = \{a^k b^k \mid k \geq 0\}$ depends on the model.

- **1-tape TM:** $O(n \log n)$
- **Multi-tape TM:** $O(n)$

Computability theory: model independence (Church-Turing Thesis)

Therefore model choice doesn't matter. Mathematically nice.

Complexity Theory: model dependence

But dependence is low (polynomial) for reasonable deterministic models.

We will focus on questions that do not depend on the model choice.

So... we will continue to use the 1-tape TM as the basic model for complexity.

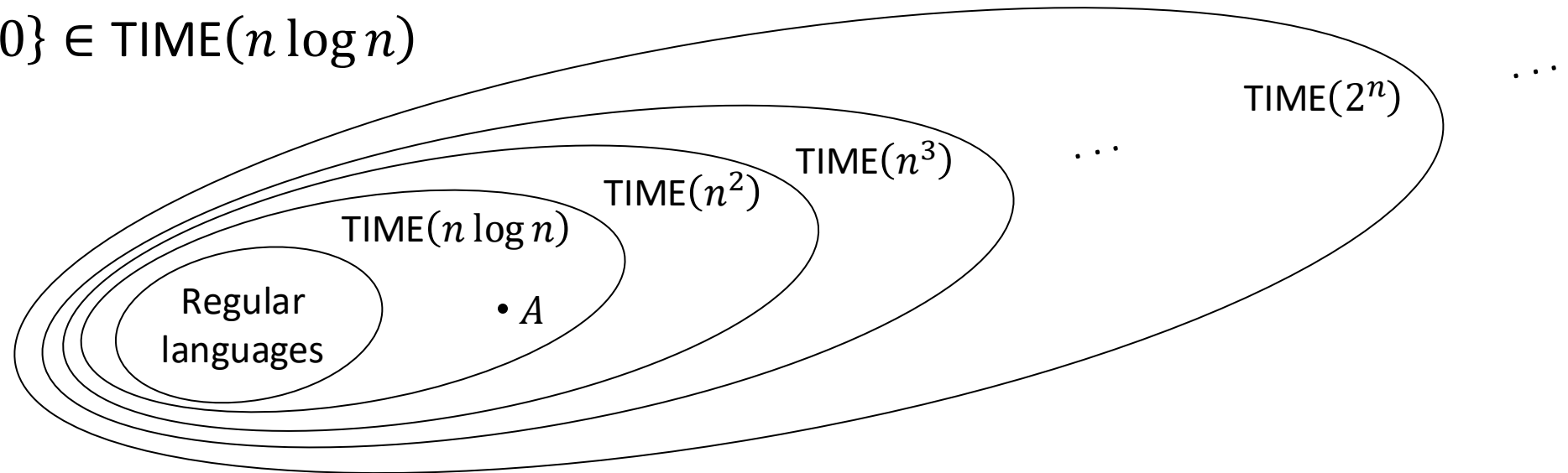
TIME Complexity Classes

Defn: Let $t: \mathbb{N} \rightarrow \mathbb{N}$. Say TM M runs in time $t(n)$ if M always halts within $t(n)$ steps on all inputs of length n .

Defn: $\text{TIME}(t(n)) = \{B \mid \text{some deterministic 1-tape TM } M \text{ decides } B \text{ and } M \text{ runs in time } O(t(n))\}$

Example:

$A = \{a^k b^k \mid k \geq 0\} \in \text{TIME}(n \log n)$



Check-in 12.2

Let $B = \{ww^R \mid w \in \{a, b\}^*\}$.

What is the smallest function t such that $B \in \text{TIME}(t(n))$?

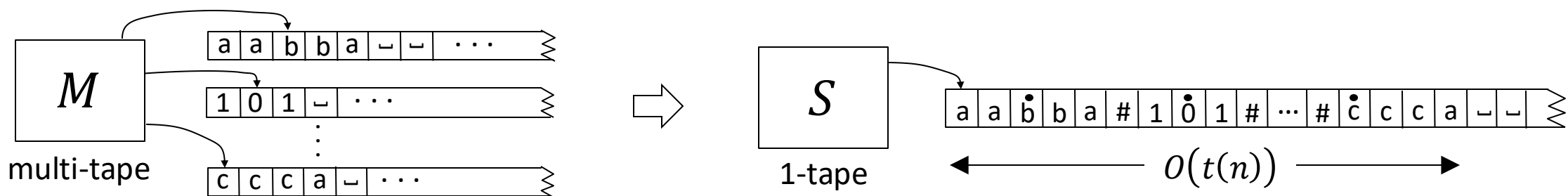
- (a) $O(n)$
- (b) $O(n \log n)$
- (c) $O(n^2)$
- (d) $O(n^3)$

Multi-tape vs 1-tape time

Theorem: Let $t(n) \geq n$.

If a multi-tape TM decides B in time $t(n)$, then $B \in \text{TIME}(t^2(n))$.

Proof: Analyze conversion of multi-tape to 1-tape TMs.



To simulate 1 step of M 's computation, S uses $O(t(n))$ steps.

So total simulation time is $O(t(n) \times t(n)) = O(t^2(n))$.

Similar results can be shown for other reasonable deterministic models.

Relationships among models

Informal Defn: Two models of computation are polynomially equivalent if each can simulate the other with a polynomial overhead:

So $t(n)$ time $\rightarrow t^k(n)$ time on the other model, for some constant k . All reasonable* deterministic models are polynomially equivalent.

- 1-tape TMs
- multi-tape TMs
- multi-dimensional TMs
- random access machine (RAM)
- cellular automata

Ignoring these differences allows us to develop a theory that is independent of the particular model of computation

Our aim is to present the fundamental properties of **computation** – **not** of Turing machines or of any other model

Obviously there is a difference between $\text{TIME}(n)$ and $\text{TIME}(n^3)$, but the difference between polynomiality and non-polynomiality is important too.

* Models that closely approximate the running time of actual computers

The Class P

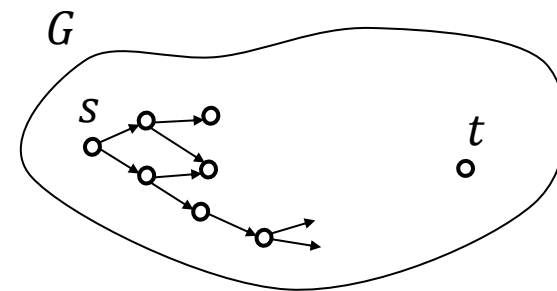
Defn: $P = \bigcup_k \text{TIME}(n^k)$
= polynomial time decidable languages

- Invariant for all reasonable deterministic models
- Corresponds roughly to realistically solvable problems
- We describe algorithms in stages
- Each stage should be polynomial
- We need to be careful with the encoding of the input:
 - An encoding is a *reasonable* encoding if:
 - It allows for polynomial time encoding and decoding of the objects into natural internal representations
 - Unary notation for numbers is not reasonable eg., 111111111111 = 12
 - The encoding is exponentially larger than in base 2 notation
 - graphs: adjacency matrix or lists are reasonable (the size is a polynomial of the number of nodes or edges) => if we show the runtime is polynomial in the number of nodes then it is polynomial in the size of the input

Example: PATH

Defn: $P = \bigcup_k \text{TIME}(n^k)$
= polynomial time decidable languages

Example: $PATH = \{ \langle G, s, t \rangle \mid G \text{ is a directed graph with a path from } s \text{ to } t \}$



Bad idea: test all possible paths (brute force algorithm):

There may be exponentially many possible paths in the number of nodes!

Example: PATH

Defn: $P = \bigcup_k \text{TIME}(n^k)$
= polynomial time decidable languages

Example: $PATH = \{\langle G, s, t \rangle \mid G \text{ is a directed graph with a path from } s \text{ to } t\}$

Theorem: $PATH \in P$

Proof: $M =$ "On input $\langle G, s, t \rangle$

1. Mark s

2. Repeat until nothing new is marked:

For each marked node x :

Scan G to mark all y where (x, y) is an edge

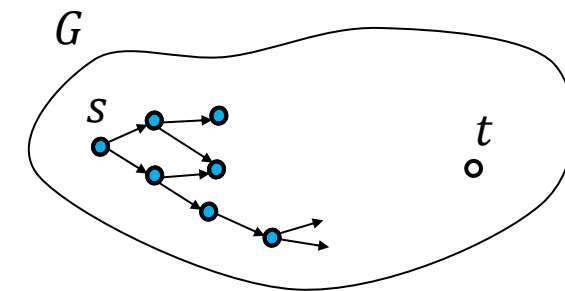
3. *Accept* if t is marked. *Reject* if not.

$\leq n$ iterations

$\times \leq n$ iterations

$\times O(n^2)$ steps

 $O(n^4)$ steps



To show polynomial time:

Each stage should be clearly polynomial and the total number of stages polynomial.

Example 2: RELPRIME

Example: $RELPRIME = \{\langle x, y \rangle \mid x \text{ and } y \text{ are relatively prime}\}$

Two numbers are **relatively prime** if 1 is the largest number that evenly divides them both
Eg. 10 and 21 are rel. prime; 10 and 22 are not

Theorem: $RELPRIME \in P$

One (wrong) Idea: brute-force algorithm

In binary the length of the number k is exponential in the length of its representation
(input length $n \Rightarrow 2^n$ different divisors to check)

Example 2: RELPRIME

Example: $RELPRIME = \{\langle x, y \rangle \mid x \text{ and } y \text{ are relatively prime}\}$

Two numbers are **relatively prime** if 1 is the largest that evenly divides them both
Eg. 10 and 21 are rel. prime; 10 and 22 are not

Theorem: $RELPRIME \in P$

Better Idea: Euclidean Greatest Common Divisor algorithm

Proof: $E =$ “On input $\langle x, y \rangle$ where x and y are binary

1. Repeat until $y = 0$:
2. Assign $x \leftarrow x \bmod y$
3. Exchange x and y
4. Output x .”

$R =$ “On input $\langle x, y \rangle$ where x and y are binary

1. Run E on $\langle x, y \rangle$
2. If the result is 1 *accept*, else *reject*

$\leq O(n)$ iterations $\rightarrow$ (Each division cuts x by at least
 $\times$ poly-time steps half, except maybe the first)

Poly -time

In total fewer iterations than
 $\max\{2 \log x, 2 \log y\}$

These logarithms are proportional to
the lengths of the representations:
 $\log 2^n = n$

Example 3: CFL

Theorem: $\text{CFL} \in \text{P}$

Idea: We proved that every context free language is decidable

Bad Idea: use that algorithm

Let $A_{\text{CFG}} = \{\langle G, w \rangle \mid G \text{ is a CFG and } w \in L(G)\}$

Theorem: A_{CFG} is decidable (lecture 9)

Proof: Give TM $D_{A-\text{CFG}}$ that decides A_{CFG} .

$D_{A-\text{CFG}}$ = “On input $\langle G, w \rangle$

1. Convert G into CNF.
2. Try all derivations of length $2|w| - 1$.
3. *Accept* if any generate w .
Reject if not.

Problem: The number of derivations with k steps, may be exponential in k

Solution: Use **Dynamic Programming** to:

1. solve smaller substrings and save the solutions to the smaller subproblems!
2. Use the solutions to the smaller subproblems to generate larger substrings!

Recall Chomsky Normal Form (CNF) only allows rules:

$A \rightarrow BC$

$B \rightarrow b$

Lemma 1: Can convert every CFG into CNF (shown)

Lemma 2: If H is in CNF and $w \in L(H)$ then every derivation of w has exactly $2|w| - 1$ steps.
Proof: exercise (solved).

Example 3: CFL.(2)

Theorem: $\text{CFL} \in \text{P}$

Idea: Use **Dynamic Programming** to:

1. solve smaller substrings and save the solutions to the smaller subproblems!
2. Use the solutions to the smaller subproblems to generate larger substrings!
 - Let $w = w_1 w_2 \dots w_n$ be the string to be generated
 - Create a table of size $n \times n$:
 - for $i \leq j$, the (i, j) th entry contains the variables that generate $w_i w_{i+1} \dots w_j$
 - First fill the entries for strings of length 1, then length 2 and so on...
 - Eg., If the algorithm has determined which variables generate all substrings of length k
 - For length $k + 1$ the algorithm splits each substring in all possible ways and checks the table
 - For $A \rightarrow BC$ check whether B generates the first bit and C generates the second
 - For length 1 the algorithm checks all the rules $A \rightarrow b$

Example 3: CFL (3)

Theorem: $\text{CFL} \in \text{P}$

$D =$ “On input $w = w_1 \cdots w_n$:

1. For $w = \epsilon$, if $S \rightarrow \epsilon$ is a rule, *accept*; else, *reject*. $\llbracket w = \epsilon \text{ case} \rrbracket$
2. For $i = 1$ to n : $\llbracket \text{examine each substring of length 1} \rrbracket$ $O(n)$ iterations (at most nv times)
3. For each variable A : $\times$ poly-time steps
4. Test whether $A \rightarrow \mathbf{b}$ is a rule, where $\mathbf{b} = w_i$.
5. If so, place A in $\text{table}(i, i)$.
6. For $l = 2$ to n : $\llbracket l \text{ is the length of the substring} \rrbracket$ $O(n)$ iterations
7. For $i = 1$ to $n - l + 1$: $\llbracket i \text{ is the start position of the substring} \rrbracket$ $\times O(n)$ iterations
8. Let $j = i + l - 1$. $\llbracket j \text{ is the end position of the substring} \rrbracket$
9. For $k = i$ to $j - 1$: $\llbracket k \text{ is the split position} \rrbracket$ $\times O(n)$ iterations
10. For each rule $A \rightarrow BC$: r times (constant as well as v above)
11. If $\text{table}(i, k)$ contains B and $\text{table}(k + 1, j)$ contains C , put A in $\text{table}(i, j)$. $\times$ poly-time steps
12. If S is in $\text{table}(1, n)$, *accept*; else, *reject*.”

 $O(n^3)$ iterations $\times$ poly-time steps

PATH and HAMPATH

Example: $HAMPATH = \{\langle G, s, t \rangle \mid G \text{ is a directed graph with a path from } s \text{ to } t \text{ and the path goes through every node of } G\}$

Recall Theorem: $PATH \in P$

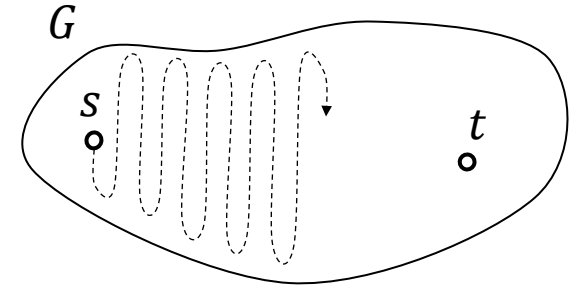
Called a Hamiltonian path

Question: $HAMPATH \in P$?

“On input $\langle G, s, t \rangle$

1. Let m be the number of nodes in G .
2. For each path of length m in G :
test if m is a Hamiltonian path from s to t .
Accept if yes.
3. Reject if all paths fail.”

May be $m! > 2^m$ paths of length m
so algorithm is exponential time
not polynomial time.



Check-in 12.3

Is $HAMPATH \in P$?

- (a) Definitely Yes. You have a polynomial-time algorithm.
- (b) Probably Yes. It should be similar to showing $PATH \in P$.
- (c) Toss up.
- (d) Probably No. Hard to beat the exponential algorithm.
- (e) Definitely No. You can prove it!

Quick review of today

1. Introduction to Complexity Theory
2. Which model to use? 1-tape-TMs
3. $\text{TIME}(t(n))$ complexity classes
4. The class P
5. $PATH \in P$
6. $RELPRIME \in P$
7. $CFL \in P$